

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
PHÁT TRIỂN ỨNG DỤNG CHO
THIẾT BỊ DI ĐỘNG (MAD)
ỨNG DỤNG PODY

*Nền tảng nghe podcast và tạo nội dung
podcast hỗ trợ AI*

Giảng viên hướng dẫn: *ThS. Nguyễn Hoàng Anh*

Nhóm QLĐT: *CNPM05*

Nhóm BTL: *01*

Thành viên thực hiện: *Vũ Đức Thành – B22DCCN801*

Danh sách thành viên:

STT	Họ và tên	MSSV
1	Lại Xuân Hiếu	B22DCCN309
2	Vũ Đức Thành	B22DCCN801
3	Nguyễn Việt Huy	B22DCCN393
4	Lê Minh Ngọc	B22DCCN586

Hà Nội, 2026

BẢNG PHÂN CÔNG THÀNH VIÊN VÀ ĐÓNG GÓP

Bảng dưới đây trình bày chi tiết phân công công việc và phạm vi phụ trách của từng thành viên trong nhóm.

Bảng 1: Phân công thành viên và đóng góp

STT	Họ và tên	MSSV	Vai trò	Phạm vi phụ trách & Công việc	%
1	Lại Xuân Hiếu	B22DCCN309	Thành viên	AI tạo podcast – ai-service, content-service. Phụ trách luồng creator tạo podcast/show bằng AI, quản lý voice profiles, chat-create, production plan, job tạo show/episode, home feed, bookmark.	25%
2	Vũ Đức Thành	B22DCCN801	Thành viên	Báo và podcast báo – article_service, embedding_service. Phụ trách crawl bài báo, category, tương tác bài viết, tạo podcast từ nhóm bài báo, embedding/search semantic, gán category ngữ nghĩa.	25%
3	Nguyễn Việt Huy	B22DCCN393	Thành viên	Hạ tầng và xác thực – api-gateway, identity-service. Phụ trách cửa vào hệ thống, routing, JWT auth, đăng ký/đăng nhập, xác minh email, password reset, outbox event.	25%
4	Lê Minh Ngọc	B22DCCN586	Thành viên	Thông báo – notification-service. Phụ trách notification inbox, unread count, settings, email hệ thống, consume Kafka event, retry/DLQ, delivery logs.	25%

MỤC LỤC

1	MỞ ĐẦU VÀ YÊU CẦU	7
1.1	Giới thiệu ứng dụng	7
1.2	Lý do chọn đề tài	7
1.3	Concept thực hiện ứng dụng	8
1.4	Phân tích yêu cầu ứng dụng	9
1.4.1	Yêu cầu chức năng	9
1.4.2	Yêu cầu phi chức năng	9
1.5	Phân tích và lựa chọn công nghệ	10
2	PHÂN TÍCH THIẾT KẾ TỔNG QUAN	12
2.1	Kiến trúc tổng quan hệ thống	12
2.1.1	Tầng Client	12
2.1.2	Tầng API Gateway	12
2.1.3	Tầng Backend Services	13
2.1.4	Tầng Infrastructure	14
2.2	Luồng xử lý tổng quan	14
2.2.1	Luồng xác thực (Authentication Flow)	14
2.2.2	Luồng nghe podcast/show (Content Flow)	15
2.2.3	Luồng tạo podcast bằng AI (AI Creator Flow)	15
2.2.4	Luồng bài báo và podcast báo (Article & Embedding Flow)	16
2.3	Use case tổng quan	17
2.3.1	Tác nhân (Actor)	17
2.3.2	Danh sách use case tổng quan	17
2.4	Mô hình dữ liệu tổng quan (ER theo cụm service)	18
2.4.1	Identity DB	18
2.4.2	Content DB	19
2.4.3	AI DB	20
2.4.4	Article DB	21
2.4.5	Embedding DB	22

2.4.6	Notification DB	22
2.5	Mô hình triển khai	23
2.5.1	Thứ tự khởi động	24
2.5.2	Kiểm tra hệ thống	25
2.6	Bảng mapping Service – Chức năng – Database – Endpoint chính	26
3	THIẾT KẾ CHI TIẾT CÁ NHÂN	27
3.1	Vũ Đức Thành – Báo và podcast báo: <code>article_service</code> và <code>embedding_service</code>	27
3.1.1	Phạm vi phụ trách	27
3.1.2	Use case chi tiết	28
3.1.3	Component diagram	29
3.1.4	Mô hình dữ liệu riêng	29
3.1.5	Sequence diagram	31
3.1.6	API/service contract	33
3.1.7	Giao diện liên quan và kết quả đạt được	33
3.1.8	Hạn chế riêng	33
3.2	Tổng hợp đầu ra Phase 3	33
4	KẾT QUẢ, THỬ NGHIỆM VÀ ĐÁNH GIÁ	34
4.1	Kết quả ứng dụng	34
4.1.1	Kết quả tổng quát cấp nhóm	34
4.1.2	Kết quả theo phạm vi cá nhân	35
4.2	Cài đặt và triển khai ứng dụng	35
4.2.1	Yêu cầu môi trường	35
4.2.2	Các bước triển khai local	35
4.2.3	URL kiểm tra nhanh	36
4.3	Kết quả thử nghiệm cá nhân	36
4.4	Số liệu minh chứng trong CSDL	37
4.5	Đánh giá hạn chế	37
4.5.1	Hạn chế chung	37
4.5.2	Hạn chế riêng của phần cá nhân	38
4.6	Định hướng phát triển	38
4.7	Tài liệu tham khảo	38

DANH SÁCH HÌNH VẼ

3.1	Component diagram cụm <code>article_service</code> và <code>embedding_service</code>	29
3.2	ER riêng cho cụm <code>Article</code> và <code>Embedding</code>	30
3.3	Sequence diagram xem chi tiết bài báo	31
3.4	Sequence diagram tạo podcast từ nhóm bài báo	32

DANH SÁCH BẢNG

1	Phân công thành viên và đóng góp	1
1.1	Yêu cầu chức năng của hệ thống Pody	9
1.2	Yêu cầu phi chức năng của hệ thống Pody	10
1.3	Bảng công nghệ và lý do lựa chọn	11
2.1	Danh sách backend service	13
2.2	Thành phần hạ tầng	14
2.3	Danh sách tác nhân hệ thống	17
2.4	Use case tổng quan hệ thống Pody	18
2.5	Các bảng trong Identity DB	19
2.6	Các bảng trong Content DB	20
2.7	Các bảng trong AI DB	21
2.8	Các bảng trong Article DB	22
2.9	Các bảng trong Embedding DB	22
2.10	Các bảng trong Notification DB	23
2.11	Mô hình triển khai Docker Compose	24
2.12	Mapping tổng hợp service	26
3.1	Use case chi tiết cụm Article và Embedding	28
3.2	Endpoint chính cụm Article và Embedding	33
3.3	Tổng hợp đầu ra Phase 3 – Article + Embedding	33
4.1	Kết quả tổng quát cấp nhóm	34
4.2	Kết quả cá nhân – Article + Embedding	35
4.3	URL kiểm tra hệ thống sau khi khởi động	36
4.4	Bảng test case cá nhân – Article + Embedding	36
4.5	Số liệu demo cá nhân – Article + Embedding	37

DANH SÁCH TỪ VIẾT TẮT

Từ viết tắt	Giải thích
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CDC	Change Data Capture – Thu thập thay đổi dữ liệu
CORS	Cross-Origin Resource Sharing – Chia sẻ tài nguyên nguồn chéo
CRUD	Create, Read, Update, Delete
DB	Database – Cơ sở dữ liệu
DLQ	Dead Letter Queue – Hàng đợi thư chết
ER	Entity-Relationship – Thực thể - Quan hệ
GCS	Google Cloud Storage
JWT	JSON Web Token
MAD	Mobile Application Development – Phát triển ứng dụng di động
REST	Representational State Transfer
RSS	Really Simple Syndication
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TTS	Text-to-Speech – Chuyển văn bản thành giọng nói
UI	User Interface – Giao diện người dùng

CHƯƠNG 1

MỞ ĐẦU VÀ YÊU CẦU

1.1 Giới thiệu ứng dụng

Pody là một nền tảng ứng dụng di động cho phép người dùng nghe podcast, đọc bài báo và tạo nội dung podcast với sự hỗ trợ của trí tuệ nhân tạo (AI). Ứng dụng được thiết kế để phục vụ hai nhóm người dùng chính:

- **Listener (Người nghe):** Có thể đăng ký/đăng nhập tài khoản, khám phá nội dung trên trang chủ, đọc và tìm kiếm bài báo theo category hoặc tìm kiếm ngữ nghĩa (semantic search), nghe podcast, lưu tập yêu thích (bookmark), tương tác với bài viết thông qua reaction/comment và theo dõi thông báo trong ứng dụng.
- **Creator (Nhà sáng tạo nội dung):** Có thể sử dụng AI để trò chuyện, lên kế hoạch sản xuất (production plan), tạo show và episode podcast một cách tự động. Creator cũng có thể chọn nhóm bài báo để tạo podcast dạng tổng hợp tin tức.

Hệ thống Pody được xây dựng theo kiến trúc **microservices**, bao gồm ứng dụng Flutter cho phía client, một API Gateway điều phối và xác thực, cùng nhiều backend service độc lập phụ trách các nghiệp vụ riêng biệt. Mỗi service có cơ sở dữ liệu riêng, giao tiếp qua REST API và sự kiện bất đồng bộ qua Apache Kafka.

1.2 Lý do chọn đề tài

Việc lựa chọn đề tài Pody xuất phát từ các lý do sau:

1. **Xu hướng tiêu thụ nội dung dạng âm thanh:** Nhu cầu nghe podcast và tiêu thụ nội dung ngắn dạng âm thanh đang tăng trưởng mạnh mẽ trên toàn cầu. Người dùng ngày càng ưa chuộng việc nghe nội dung

trong lúc di chuyển, tập thể dục hay làm việc nhà thay vì đọc bài viết dài.

2. **Nhu cầu tóm tắt và chuyển đổi nội dung:** Tin tức và nội dung dài cần được tóm tắt, chuyển đổi thành dạng dễ nghe hơn. Podcast là phương tiện phù hợp để truyền tải thông tin một cách ngắn gọn và sinh động.
3. **AI hỗ trợ sáng tạo nội dung:** Trí tuệ nhân tạo có thể hỗ trợ lập kế hoạch sản xuất, tạo kịch bản (script), tạo transcript và audio thông qua TTS (Text-to-Speech), giúp rút ngắn đáng kể thời gian sản xuất podcast so với quy trình truyền thống.
4. **Phù hợp để áp dụng nhiều công nghệ hiện đại:** Đề tài là môi trường lý tưởng để áp dụng đồng thời nhiều công nghệ: mobile app (Flutter), backend microservices (Go, Python/FastAPI), AI/ML (Google GenAI, embedding, TTS), tìm kiếm ngữ nghĩa (pgvector), kiến trúc event-driven (Kafka, Debezium), hệ thống thông báo và nhiều hơn nữa.

1.3 Concept thực hiện ứng dụng

Concept cốt lõi của Pody được tóm gọn trong câu:

“Đọc nhanh, nghe sau, tạo podcast bằng AI”

Hệ thống Pody vận hành theo hai luồng chính:

- **Luồng Listener:** Người dùng khám phá nội dung trên trang chủ, đọc bài báo theo chủ đề, nghe podcast từ các show đa dạng, lưu bookmark episode yêu thích, nhận thông báo về nội dung mới và tương tác với cộng đồng qua reaction/comment.
- **Luồng Creator:** Nhà sáng tạo nội dung trò chuyện với AI để lên ý tưởng, tạo production plan chi tiết cho show podcast, sau đó hệ thống tự động tạo show/episode hoàn chỉnh bao gồm script, audio và transcript. Creator cũng có thể quản lý các show đã tạo thông qua giao diện “My Shows”.

1.4 Phân tích yêu cầu ứng dụng

1.4.1 Yêu cầu chức năng

Bảng 1.1 trình bày các yêu cầu chức năng chính của hệ thống Pody.

Bảng 1.1: Yêu cầu chức năng của hệ thống Pody

Mã	Nhóm chức năng	Mô tả
FR-01	Xác thực người dùng	Đăng ký tài khoản, đăng nhập bằng email/password, đăng nhập bằng Google, xác minh email, quên mật khẩu và đặt lại mật khẩu.
FR-02	Quản lý show & episode	Xem danh sách show trên trang chủ, xem chi tiết show, xem danh sách episode trong show, xem chi tiết episode (audio, transcript).
FR-03	Bookmark	Người dùng có thể lưu (bookmark) hoặc bỏ lưu episode yêu thích để nghe lại sau.
FR-04	Đọc bài báo	Xem danh sách bài báo, lọc theo category, tìm kiếm bài báo, xem chi tiết bài báo.
FR-05	Tương tác bài báo	Reaction (like/dislike/...) bài báo, bình luận (comment) bài báo, hệ thống ghi nhận metric đọc bài.
FR-06	Tạo podcast từ bài báo	Người dùng chọn nhóm bài báo, hệ thống tạo job nền để tổng hợp nội dung, viết script, tạo audio/transcript.
FR-07	Tạo show bằng AI	Creator trò chuyện với AI (chat-create), tạo production plan, tạo show/episode từ plan. AI tự động sinh script, TTS, transcript.
FR-08	Thông báo	Nhận thông báo trong ứng dụng (in-app notification), xem danh sách thông báo, đếm unread, đánh dấu đã đọc, cài đặt thông báo. Hệ thống gửi email xác minh và email reset password.

1.4.2 Yêu cầu phi chức năng

Bảng 1.2 trình bày các yêu cầu phi chức năng mà hệ thống Pody cần đáp ứng.

Bảng 1.2: Yêu cầu phi chức năng của hệ thống Pody

Mã	Yêu cầu	Mô tả
NFR-01	Kiến trúc microservices	Hệ thống được tách thành các service độc lập, mỗi service đảm nhận một nghiệp vụ riêng biệt, dễ bảo trì và mở rộng.
NFR-02	API Gateway	Có một cổng API duy nhất (API Gateway) điều phối request đến các service, thực hiện kiểm tra JWT và quản lý routing.
NFR-03	Tách biệt dữ liệu	Mỗi service có database riêng (database-per-service), không phụ thuộc foreign key vật lý chéo service. Tham chiếu giữa các service thông qua external ID.
NFR-04	Health check & Docs	Mỗi service cung cấp endpoint health check (/healthz) và tài liệu API tự động (OpenAPI/Swagger docs).
NFR-05	Kiểm thử	Hệ thống có test coverage cho các API endpoint, gateway routing, event processing và docs.
NFR-06	Triển khai local	Toàn bộ hệ thống có thể triển khai trên môi trường local thông qua Docker Compose để phục vụ demo và phát triển.
NFR-07	Event-driven	Giao tiếp bất đồng bộ giữa các service thông qua Apache Kafka, đảm bảo loose coupling và khả năng retry/DLQ khi xử lý event.

1.5 Phân tích và lựa chọn công nghệ

Bảng 1.3 tổng hợp các công nghệ được sử dụng trong hệ thống Pody cùng lý do lựa chọn.

Bảng 1.3: Bảng công nghệ và lý do lựa chọn

Lĩnh vực	Công nghệ	Lý do lựa chọn
Mobile App	Flutter	Framework đa nền tảng (iOS/Android) với hiệu năng cao, hot reload nhanh, hệ sinh thái widget phong phú, phù hợp cho việc xây dựng giao diện người dùng hiện đại.
API Gateway	Go, chi router	Go có hiệu năng cao, biên dịch nhanh, phù hợp cho reverse proxy. Router chi nhẹ, hỗ trợ middleware linh hoạt cho JWT auth, CORS, logging, request ID.
Backend (Go)	Go, chi	Sử dụng cho identity-service, content-service, notification-service. Go phù hợp cho service backend nhờ goroutine, quản lý concurrent tốt.
Backend (Python)	Python, FastAPI	Sử dụng cho ai-service, article_service, embedding_service. FastAPI hiệu năng cao, tự động sinh OpenAPI docs, phù hợp cho tích hợp AI/ML.
Database	PostgreSQL	RDBMS mạnh mẽ, ổn định, hỗ trợ JSON, full-text search. Mỗi service có instance PostgreSQL riêng.
Vector DB	PostgreSQL + pgvector	Extension pgvector cho phép lưu trữ và tìm kiếm vector embedding trực tiếp trong PostgreSQL, không cần database vector riêng.
Message Broker	Apache Kafka	Nền tảng streaming phân tán, đảm bảo thứ tự message, hỗ trợ consumer group, retry topic và DLQ cho event-driven architecture.
CDC	Debezium	Capture thay đổi dữ liệu từ Article DB và publish event lên Kafka topic, giúp Embedding Service nhận biết bài báo mới để tạo embedding tự động.
Cache	Redis	In-memory data store, dùng làm cache phụ trợ cho các service, hỗ trợ tốc độ truy xuất nhanh.
Object Storage	MinIO / Google Cloud Storage	Lưu trữ file media (audio, transcript, ảnh). MinIO tương thích S3 API, phù hợp cho môi trường local. GCS cho production.
AI Provider	Google GenAI (Gemini)	Sử dụng Gemini cho chat/generation, embedding model cho vector search, TTS cho chuyển văn bản thành giọng nói.
Triển khai	Docker Compose	Orchestration tất cả service, database, Kafka, Redis, MinIO trên môi trường local bằng một lệnh duy nhất.

CHƯƠNG 2

PHÂN TÍCH THIẾT KẾ TỔNG QUAN

Chương này trình bày thiết kế tổng quan của hệ thống Pody ở mức nhóm, bao gồm kiến trúc hệ thống, các luồng xử lý chính, use case tổng quan, mô hình dữ liệu tổng quan theo cụm service và mô hình triển khai.

2.1 Kiến trúc tổng quan hệ thống

Hệ thống Pody được xây dựng theo kiến trúc **microservices**, trong đó mỗi service đảm nhận một nghiệp vụ riêng biệt, có database độc lập và giao tiếp thông qua REST API hoặc sự kiện bất đồng bộ qua Apache Kafka.

Kiến trúc tổng quan bao gồm các tầng sau:

2.1.1 Tầng Client

- **Flutter App:** Ứng dụng di động đa nền tảng (iOS/Android) là giao diện duy nhất để người dùng tương tác với hệ thống. Mọi request từ app đều đi qua API Gateway.

2.1.2 Tầng API Gateway

- **API Gateway** (Go, chi router) – cổng vào duy nhất của hệ thống, đảm nhận:
 - Reverse proxy request đến các backend service.
 - Phân biệt route public (`/api/v1/public/*`) và route protected (`/api/v1/*`).
 - JWT authentication middleware cho route protected.
 - Bổ sung request ID, logging, recovery middleware và CORS.
 - Cung cấp route metadata (`/api/v1/_meta/routes`) và tổng hợp docs.

2.1.3 Tầng Backend Services

Hệ thống gồm 6 backend service độc lập:

Bảng 2.1: Danh sách backend service

Service	Ngôn ngữ	Chức năng chính
identity-service	Go	Xác thực người dùng: sign-up, sign-in, Google sign-in, email verification, password reset, user profile, outbox event.
content-service	Go	Quản lý nội dung: show, episode, host, category, tag, home feed, bookmark, episode assets & segments.
ai-service	Python/FastAPI	AI tạo podcast: voice profiles, chat-create threads, production plans, generation jobs, TTS, transcript alignment.
article_service	Python/FastAPI	Bài báo: crawl RSS, articles, categories, reactions, comments, metrics, article podcast jobs.
embedding_service	Python/FastAPI	Embedding & search: consume CDC event, chunk text, tạo embedding vector, semantic search, gán category ngữ nghĩa.
notification-service	Go	Thông báo: notification inbox, unread count, settings, email sending, consume Kafka event, retry/DLQ, delivery logs.

2.1.4 Tầng Infrastructure

Bảng 2.2: Thành phần hạ tầng

Thành phần	Mô tả
PostgreSQL (nhiều instance)	Mỗi service có database riêng: Identity DB, Content DB, AI DB, Article DB, Notification DB.
PostgreSQL + pgvector	Embedding DB với extension pgvector để lưu trữ và tìm kiếm vector embedding.
Apache Kafka	Message broker cho giao tiếp event-driven giữa các service (verification, password reset, CDC, category sync).
Debezium Connect	CDC connector capture thay đổi từ Article DB và publish lên Kafka topic để Embedding Service xử lý.
Redis	Cache phụ trợ cho Article Service (crawler interval, session).
MinIO / GCS	Object storage cho media: audio, transcript, ảnh avatar, cover image. MinIO cho local, GCS cho production.
Kafka UI	Giao diện web quản lý Kafka topics, consumers, connectors (chỉ dùng trong dev/local).

2.2 Luồng xử lý tổng quan

Phần này mô tả 4 luồng xử lý chính trong hệ thống Pody.

2.2.1 Luồng xác thực (Authentication Flow)

1. Người dùng gửi yêu cầu đăng ký/đăng nhập từ Flutter App.
2. Request đi qua API Gateway (route `public /api/v1/public/identity/*`)
3. API Gateway proxy request đến `identity-service`.
4. `identity-service` xử lý:
 - **Đăng ký:** Tạo user, tạo email verification token, ghi outbox event.
 - **Đăng nhập:** Kiểm tra credentials, trả access token + refresh token.
 - **Google sign-in:** Xác thực Google ID token, tạo/liên kết user.
5. Outbox publisher poll outbox event và publish lên Kafka topic `identity.email.verification.requested`.

6. `notification-service` consume event, kiểm tra idempotency, gửi email xác minh.
7. Người dùng bấm link verify → `identity-service` cập nhật `email_verified`.

2.2.2 Luồng nghe podcast/show (Content Flow)

1. Người dùng mở app, Flutter gọi API home feed qua API Gateway.
2. API Gateway proxy đến `content-service`.
3. `content-service` truy vấn Content DB:
 - Lấy danh sách show (published, sorted by updated).
 - Lấy danh sách episode mới nhất.
 - Lấy thông tin host, category, tag.
4. Khi người dùng chọn xem chi tiết show/episode, Flutter gọi API show detail/episode detail.
5. `content-service` trả thông tin chi tiết bao gồm audio URL, transcript (episode segments), cover image.
6. Người dùng có thể bookmark episode → `content-service` lưu vào bảng `episode_bookmarks`.

2.2.3 Luồng tạo podcast bằng AI (AI Creator Flow)

1. Creator mở giao diện AI Setup, Flutter gọi API tạo chat thread qua API Gateway.
2. API Gateway xác thực JWT, proxy đến `ai-service`.
3. Creator gửi message để refine ý tưởng podcast → `ai-service` gọi Google GenAI (Gemini) để tạo phản hồi.
4. Khi ý tưởng đã rõ, AI tạo production plan bao gồm: series title, description, hosts, episode drafts, sources.

5. Creator xác nhận tạo show → ai-service tạo generation jobs:

- Job `show_creation`: Tạo show và hosts trong Content DB.
- Job `script_generation`: Gọi Gemini sinh script cho episode.
- Job `audio_generation`: Gọi TTS (Gemini TTS) tạo audio.
- Job `transcript_generation`: Alignment transcript với audio.
- Job `publish_episode`: Ghi episode, segments, assets vào Content DB.

6. Khi hoàn thành, ai-service gọi internal API của notification-service để tạo in-app notification cho creator.

2.2.4 Luồng bài báo và podcast báo (Article & Embedding Flow)

1. `article_service` tự động crawl bài báo từ RSS sources theo lịch (mỗi 30 phút).

2. Bài báo mới được lưu vào Article DB (bảng `articles`, `news_sources`).

3. Debezium CDC monitor thay đổi trên Article DB, publish event lên Kafka topic.

4. `embedding_service` consume CDC event:

- Chunk nội dung bài báo thành các đoạn ngắn.
- Gọi Gemini Embedding Model tạo vector embedding cho mỗi chunk.
- Lưu embedding vào Embedding DB (PostgreSQL + pgvector).
- Gán category ngữ nghĩa cho bài báo dựa trên cosine similarity với category embeddings.
- Publish kết quả gán category lên Kafka topic `article.category.mat`

5. `article_service` consume category sync event, cập nhật bảng `category_articles`.

6. Người dùng có thể:

- Đọc danh sách bài báo (có lọc theo category).
- Tìm kiếm semantic qua `embedding_service`.
- Tương tác: reaction, comment, metric đọc bài.
- Tạo article podcast job → worker tổng hợp nội dung, viết script, TTS, upload audio/transcript.

2.3 Use case tổng quan

2.3.1 Tác nhân (Actor)

Hệ thống Pody có 5 tác nhân chính:

Bảng 2.3: Danh sách tác nhân hệ thống

STT	Tác nhân	Mô tả
1	Khách (Guest)	Người dùng chưa đăng nhập, có thể xem nội dung public.
2	Người dùng (User)	Người dùng đã đăng nhập, có thể tương tác với nội dung, bookmark, nhận thông báo.
3	Creator	Người dùng với account type creator/hybrid, có thể tạo podcast bằng AI.
4	Hệ thống AI	Google GenAI (Gemini), TTS, Embedding – xử lý tự động sinh nội dung.
5	Hệ thống thông báo	Notification Service – gửi email, thông báo in-app tự động.

2.3.2 Danh sách use case tổng quan

Bảng 2.4 liệt kê các use case tổng quan của hệ thống.

Bảng 2.4: Use case tổng quan hệ thống Pody

Mã	Use case	Tác nhân	Service liên quan
UC-01	Đăng ký tài khoản	Guest	identity-service
UC-02	Xác minh email	User, Hệ thống TBáo	identity, notification
UC-03	Đăng nhập	Guest	identity-service
UC-04	Đăng nhập Google	Guest	identity-service
UC-05	Quên/Reset mật khẩu	User, Hệ thống TBáo	identity, notification
UC-06	Xem trang chủ	Guest, User	content-service
UC-07	Xem show/episode	Guest, User	content-service
UC-08	Bookmark episode	User	content-service
UC-09	Đọc bài báo	Guest, User	article_service
UC-10	Tương tác bài báo	User	article_service
UC-11	Tìm kiếm semantic	User	embedding_service
UC-12	Tạo podcast từ báo	User	article_service
UC-13	Chat với AI	Creator	ai-service
UC-14	Tạo show từ plan	Creator, Hệ thống AI	ai-service, content
UC-15	Xem thông báo	User	notification-service
UC-16	Cài đặt thông báo	User	notification-service

2.4 Mô hình dữ liệu tổng quan (ER theo cụm service)

Vì mỗi service có database riêng (database-per-service pattern), mô hình ER được trình bày theo từng cụm. Giữa các database không có foreign key vật lý chéo service – chỉ tham chiếu bằng external ID (ví dụ: `user_id`, `show_id`, `article_id`).

2.4.1 Identity DB

Database của `identity-service`, quản lý thông tin người dùng và xác thực.

Bảng 2.5: Các bảng trong Identity DB

Bảng	Mô tả
users	Thông tin người dùng: email, display name, username, avatar, account type, status, locale.
user_identities	Liên kết tài khoản bên thứ ba (Google, Apple): provider, provider_user_id.
user_devices	Thiết bị người dùng: platform (ios/android/web), push token.
auth_sessions	Phiên đăng nhập: refresh token hash, thời hạn, trạng thái revoke.
email_verification_tokens	Token xác minh email: token hash, thời hạn, trạng thái sử dụng.
password_reset_tokens	Token đặt lại mật khẩu: token hash, thời hạn.
outbox_events	Transactional outbox pattern: event chờ publish lên Kafka.
inbox_processed_events	Idempotency: theo dõi event đã xử lý để tránh xử lý trùng.

2.4.2 Content DB

Database của content-service, quản lý nội dung podcast.

Bảng 2.6: Các bảng trong Content DB

Bảng	Mô tả
categories	Danh mục nội dung: slug, name, icon, color, applies_to (show/news/mixed).
tags	Thẻ gắn: slug, name.
shows	Show podcast: title, slug, content type, language, cover image, publish status, visibility, monetization.
show_categories	Liên kết show – category (nhiều-nhiều), có is_primary.
show_tags	Liên kết show – tag (nhiều-nhiều).
show_hosts	Host của show: display name, avatar, role (host/co_host/guest/narrator), persona type (human/ai).
episodes	Episode trong show: title, slug, audio URL, duration, publish status, listen/like/comment count.
episode_tags	Liên kết episode – tag.
episode_assets	Tài nguyên đính kèm episode: transcript, waveform, subtitle, cover, attachment.
episode_segments	Đoạn transcript phân theo speaker: segment index, start/end ms, text content.
episode_bookmarks	Bookmark episode của user: user_id + episode_id.
episode_companion_blocks	Nội dung bổ trợ: timeline, quiz, takeaway, flashcards, chapter list.

2.4.3 AI DB

Database của ai-service, quản lý dữ liệu AI và quy trình tạo podcast.

Bảng 2.7: Các bảng trong AI DB

Bảng	Mô tả
voice_profiles	Hồ sơ giọng nói: provider (google/openai/elevenlabs), voice ID, gender, language, sample audio.
chat_threads	Luồng chat với AI: owner user, title, status.
chat_messages	Tin nhắn trong thread: role (user/assistant/system), text content, plan reference.
chat_attachments	File đính kèm tin nhắn: image, document, audio.
production_plans	Kế hoạch sản xuất: series title, description, tone, status, target show.
production_plan_hosts	Host trong plan: voice profile, role, persona type.
production_plan_sources	Nguồn tham khảo: article ID, title, publisher, URL, summary.
production_plan_tags	Thẻ gắn cho plan.
production_plan_episode_drafts	Bản nháp episode: title, description, estimated duration, status.
generation_jobs	Job tạo nội dung: plan/script/audio/image/transcript/show_creation/publish, status, input/output payload.

2.4.4 Article DB

Database của `article_service`, quản lý bài báo và tương tác.

Bảng 2.8: Các bảng trong Article DB

Bảng	Mô tả
news_sources	Nguồn tin RSS: domain, name, RSS URL, reliability score, status.
articles	Bài báo: title, author, summary, content, original URL, thumbnail, published_at.
categories	Danh mục bài báo: slug, name, description.
category_articles	Liên kết article – category: assignment source (manual/semantic), is_primary, score, model.
category_users	Theo dõi category của user.
article_stats	Thống kê bài báo: view count.
article_interactions	Tương tác: user – article – interaction type (LIKE, LOVE, ...).
article_comments	Bình luận bài báo: user, content.
article_metrics	Metric đọc bài: reading time seconds.

2.4.5 Embedding DB

Database của `embedding_service`, lưu trữ vector embedding và kết quả gán category.

Bảng 2.9: Các bảng trong Embedding DB

Bảng	Mô tả
article_embedding_documents	Document gốc để embedding: article_id, title, content hash, status.
article_embedding_chunks	Chunk text: document reference, chunk index, text content, char range.
article_chunk_embeddings	Vector embedding của chunk: embedding vector (pgvector), model, version.
article_document_embeddings	Vector embedding tổng hợp của document.
category_embeddings	Embedding vector cho mỗi category (dùng để gán category ngữ nghĩa).
article_category_matches	Kết quả matching article – category: similarity score, rank.
embedding_jobs	Job tạo embedding: status, article reference, error tracking.

2.4.6 Notification DB

Database của `notification-service`, quản lý thông báo và gửi email.

Bảng 2.10: Các bảng trong Notification DB

Bảng	Mô tả
notifications	Thông báo in-app: user_id, actor, type, target, title, body, is_read.
user_notification_settings	Cài đặt thông báo: push/email/new episode/comment/follow/marketing enabled.
delivery_logs	Log gửi thông báo: channel (email/push), status, provider message ID, error.
templates	Template thông báo: code, channel, title/body template.
outbox_events	Outbox pattern cho event nội bộ.
inbox_processed_events	Idempotency: event đã xử lý.

Lưu ý quan trọng: Giữa các database không có foreign key vật lý chéo service. Ví dụ, bảng `notifications` có trường `user_id` nhưng đây chỉ là external ID tham chiếu đến `users.id` trong Identity DB, không phải foreign key constraint.

2.5 Mô hình triển khai

Toàn bộ hệ thống Pody được triển khai trên môi trường local thông qua **Docker Compose**. Bảng 2.11 mô tả chi tiết các container và port mapping.

Bảng 2.11: Mô hình triển khai Docker Compose

Container	Port	Mô tả
api-gateway	8080	Cổng vào hệ thống. Reverse proxy, JWT auth, CORS.
identity-service	8081	Xác thực, user profile, outbox publisher.
content-service	8082	Show, episode, bookmark, home feed.
article-service	8084	Bài báo, category, reaction, comment, crawler, podcast job.
ai-service	8085	Chat-create, production plan, generation jobs, TTS.
notification-service	8087	Notification inbox, email sender, Kafka consumer.
embedding-service	8088	Embedding, semantic search, category matching.
article-postgres	5433	PostgreSQL cho Article Service (WAL logical cho CDC).
embedding-postgres	5434	PostgreSQL + pgvector cho Embedding Service.
migrations	–	Init-container: chạy migration cho Identity, Content, AI, Notification DB.
kafka	9092	Apache Kafka 4.0 (KRaft mode, không cần Zookeeper).
debezium-connect	8083	Debezium CDC connector cho Article DB.
kafka-ui	8090	Giao diện web quản lý Kafka.
redis	6379	Redis 7 Alpine – cache phụ trợ.
minio	9000/9001	Object storage (S3-compatible). Port 9001 cho MinIO Console.

2.5.1 Thứ tự khởi động

Docker Compose quản lý thứ tự khởi động thông qua `depends_on` với health check:

1. **Infrastructure:** Kafka, Redis, MinIO, PostgreSQL instances khởi động trước.
2. **Database init:** `migrations` (Identity/Content/AI/Notification DB) và `article-db-init`, `embedding-db-init` chạy migration/seed.
3. **CDC:** Debezium Connect khởi động sau Kafka và Article Postgres.

debezium-register đăng ký connector.

4. **Backend services:** Các service khởi động sau khi database sẵn sàng, mỗi service expose health check endpoint `/healthz`.
5. **API Gateway:** Khởi động cuối cùng, sau khi tất cả backend service healthy.

2.5.2 Kiểm tra hệ thống

Sau khi hệ thống khởi động, có thể kiểm tra qua các URL:

- Health check: `http://localhost:8080/healthz`
- Tổng hợp docs: `http://localhost:8080/docs`
- Route metadata: `http://localhost:8080/api/v1/_meta/routes`
- Identity docs: `http://localhost:8080/api/v1/public/identity`
- Content docs: `http://localhost:8080/api/v1/public/content`
- AI docs: `http://localhost:8080/api/v1/public/ai/docs`
- Notification docs: `http://localhost:8080/api/v1/public/notif`
- Kafka UI: `http://localhost:8090`
- MinIO Console: `http://localhost:9001`

2.6 Bảng mapping Service – Chức năng – Database – Endpoint chính

Bảng 2.12: Mapping tổng hợp service

Service	Database	Chức năng chính	Endpoint tiêu biểu
identity	Identity DB	Auth, profile, outbox	POST /public/identity/sign-up, sign-in, google, refresh, GET /identity/me
content	Content DB	Show, episode, bookmark	GET /public/content/home, shows/{id}, episodes/{id}, POST /content/bookmarks
ai	AI DB	Chat, plan, jobs	POST /ai/chat-create/threads, threads/{id}/messages, POST /ai/plans/{id}/create-show
article	Article DB	Báo, category, podcast	GET /article, /article/{id}, POST /article/{id}/reactions, POST /article/podcast-jobs
embedding	Embedding DB	Embedding, search	POST /public/embedding/articles/search
notification	Notification DB	Thông báo, email	GET /notifications, /unread-count, PATCH /{id}/read, PUT /settings

CHƯƠNG 3

THIẾT KẾ CHI TIẾT CÁ NHÂN

Chương này trình bày phần thiết kế chi tiết theo phạm vi phụ trách của từng thành viên. Mỗi tiểu mục bám theo cùng một cấu trúc: phạm vi service, chức năng chính, use case chi tiết, mô hình component, mô hình dữ liệu, sequence diagram, API contract, giao diện liên quan, kết quả đạt được và hạn chế.

3.1 Vũ Đức Thành – Báo và podcast báo: **article_service** và **embedding_service**

3.1.1 Phạm vi phụ trách

Vũ Đức Thành phụ trách luồng đọc báo, crawl bài báo, category, tương tác bài viết, podcast từ nhóm bài báo, embedding và semantic search. Hai service chính là:

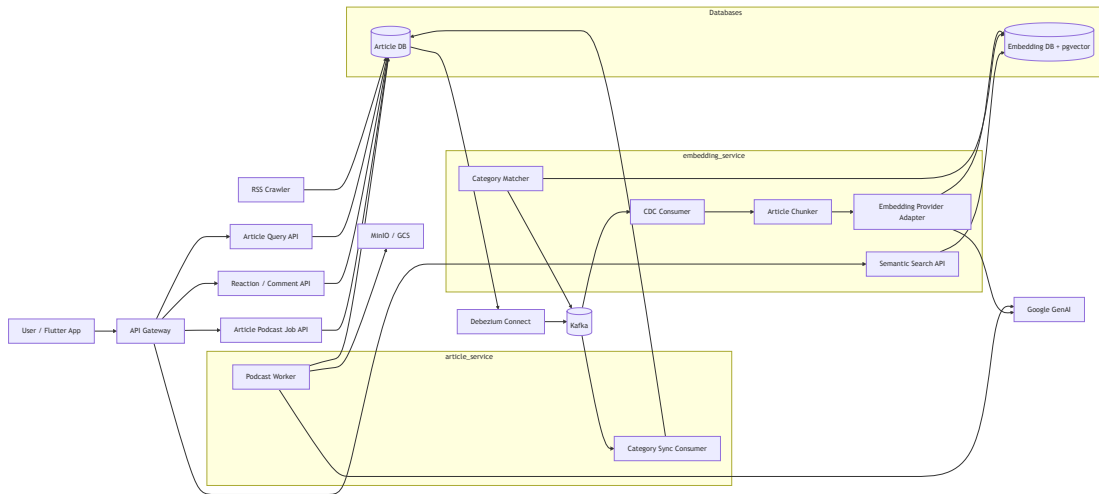
- **article_service:** Crawl RSS, quản lý bài báo, category, reaction, comment, metric đọc bài và article podcast job.
- **embedding_service:** Consume CDC event từ Kafka, chunk nội dung, gọi embedding provider, lưu vector bằng pgvector, semantic search và gán category ngữ nghĩa.

3.1.2 Use case chi tiết

Bảng 3.1: Use case chi tiết cụm Article và Embedding

Mã	Use case	Tác nhân	Mô tả
AR-UC-01	Xem danh sách bài báo	User	Lấy danh sách bài báo, có phân trang và lọc theo category/query.
AR-UC-02	Xem chi tiết bài báo	User	Lấy nội dung bài báo, tăng view count và trả reaction/comment summary.
AR-UC-03	Reaction/comment bài báo	User	Người dùng tương tác với bài báo, service lưu interaction/comment và cập nhật thống kê.
AR-UC-04	Tạo podcast từ bài báo	User	Người dùng chọn nhiều bài báo, tạo job nền để tổng hợp script, audio và transcript.
AR-UC-05	Tạo embedding khi có bài mới	Hệ thống	Debezium phát event, embedding service consume và tạo vector cho article.
AR-UC-06	Tìm kiếm semantic	User	User nhập query, service embedding tìm các bài liên quan theo độ tương đồng vector.

3.1.3 Component diagram

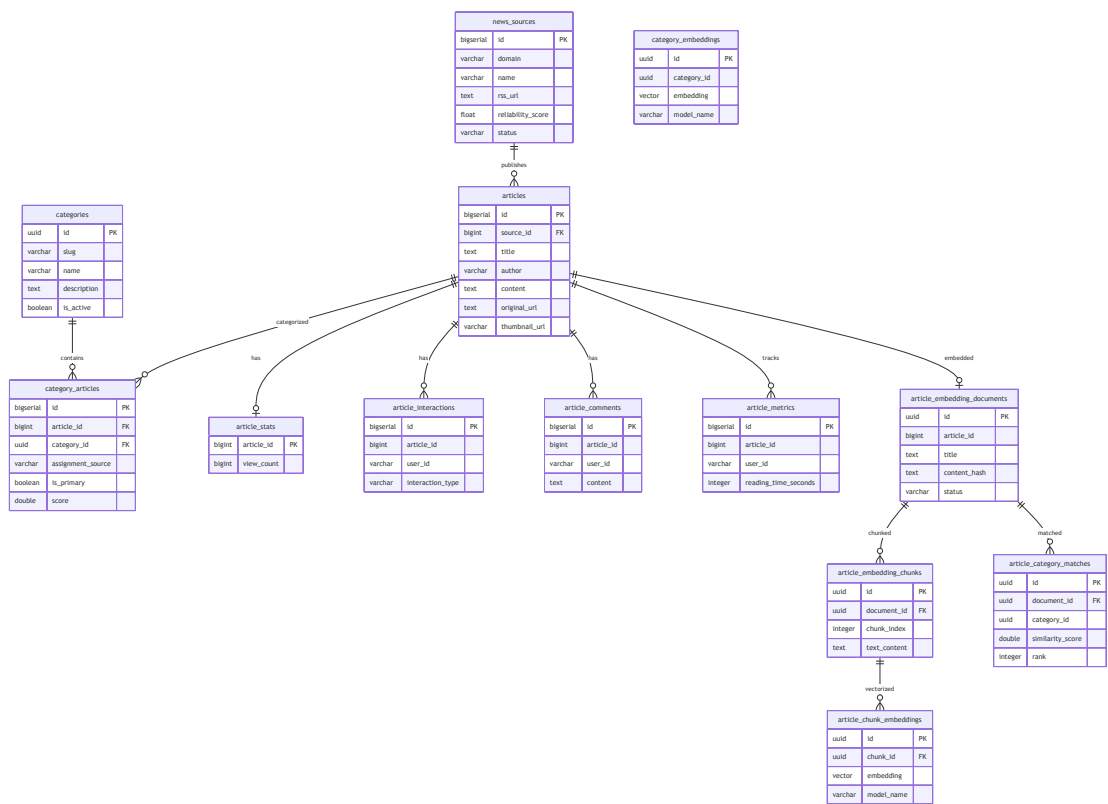


Hình 3.1: Component diagram cụm `article_service` và `embedding_service`

3.1.4 Mô hình dữ liệu riêng

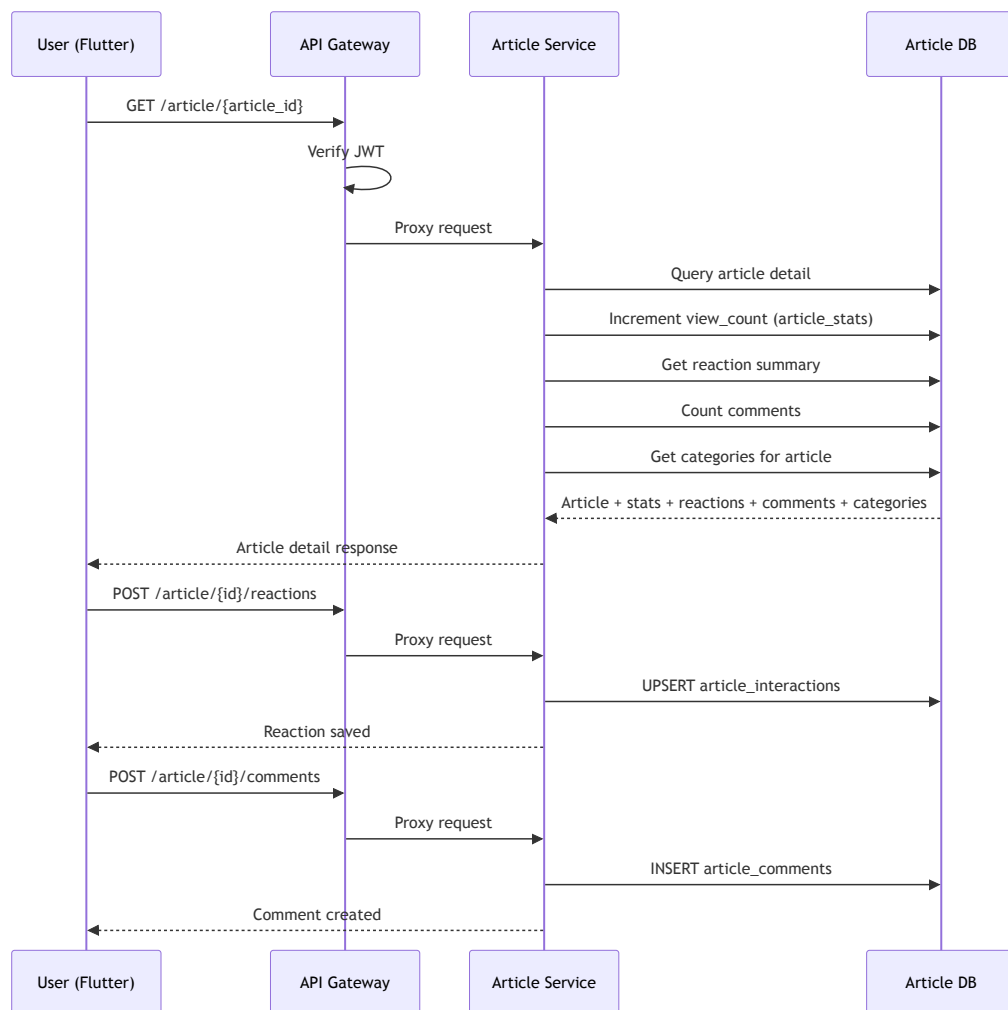
Article DB quản lý bài báo và tương tác qua các bảng `articles`, `categories`, `category_articles`, `article_stats`, `article_interactions`,

article_comments, article_metrics, article_podcast_jobs, article_podcast_job_articles, article_podcast_drafts và article_podcast_assets. Embedding DB lưu document/chunk/vector/job và kết quả category match.

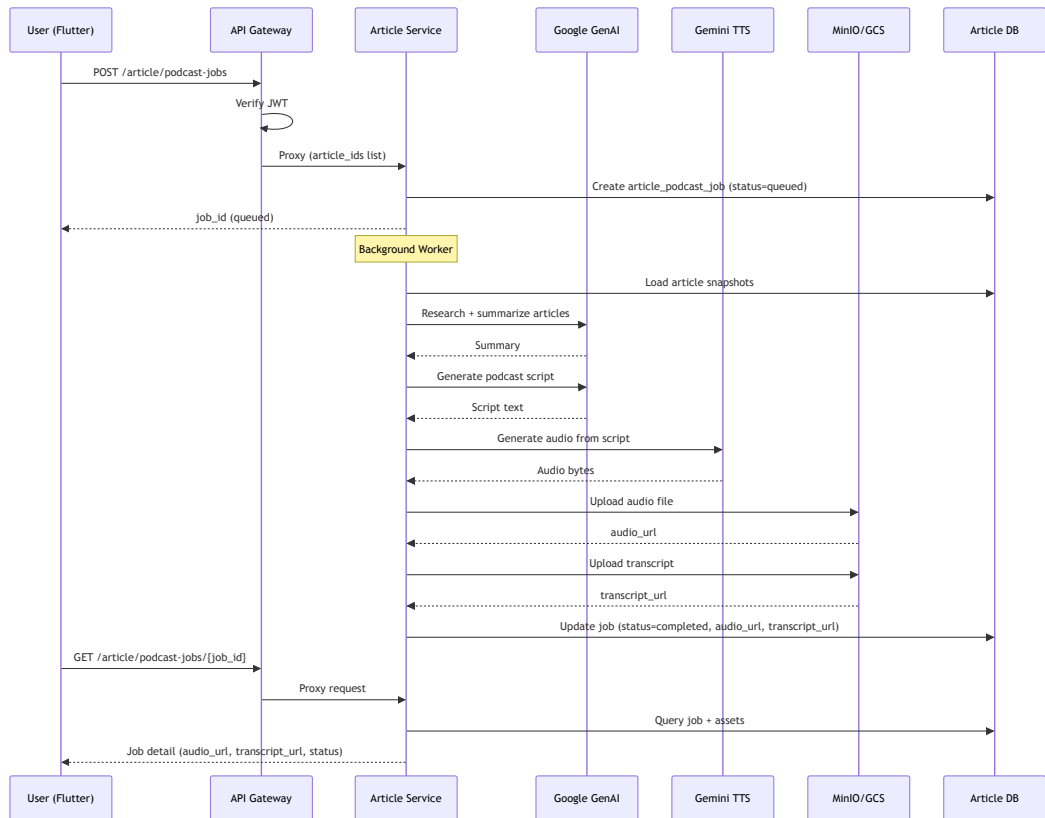


Hình 3.2: ER riêng cho cụm Article và Embedding

3.1.5 Sequence diagram



Hình 3.3: Sequence diagram xem chi tiết bài báo



Hình 3.4: Sequence diagram tạo podcast từ nhóm bài báo

3.1.6 API/service contract

Bảng 3.2: Endpoint chính cụm Article và Embedding

Method	Endpoint	Mục đích
GET	/api/v1/article	Lấy danh sách bài báo.
GET	/api/v1/article/categories	Lấy danh sách category bài báo.
GET	/api/v1/article/{article_id}	Lấy chi tiết bài báo.
POST	/api/v1/article/{article_id}/reactions	Ghi reaction của user.
GET	/api/v1/article/{article_id}/comments	Lấy danh sách comment.
POST	/api/v1/article/podcast-jobs	Tạo job podcast từ nhóm bài báo.
GET	/api/v1/article/podcast-jobs/{job_id}	Theo dõi trạng thái job podcast.
POST	/api/v1/public/embedding/articles/search	Tìm kiếm semantic theo query.

3.1.7 Giao diện liên quan và kết quả đạt được

Các giao diện liên quan gồm news list, article detail, chọn bài để tạo podcast, article podcast job/library/playback và kết quả semantic search. Cụm service đã tách rõ dữ liệu bài báo với dữ liệu embedding; Debezium/Kafka giúp phát hiện bài mới để tạo vector và đồng bộ category ngữ nghĩa về Article DB.

3.1.8 Hạn chế riêng

Chất lượng search và category matching phụ thuộc chất lượng nội dung crawl, prompt/chunking và embedding model. Podcast từ bài báo cần xử lý nền vì các bước research, script, TTS và upload audio có độ trễ cao.

3.2 Tổng hợp đầu ra Phase 3

Bảng 3.3 tổng hợp đầu ra của phần cá nhân trong Chương 3.

Bảng 3.3: Tổng hợp đầu ra Phase 3 – Article + Embedding

Cụm phụ trách	Sơ đồ đã bổ sung	Nội dung chính
Article + Embedding	Component, ER, sequence article detail, sequence article podcast	Crawl/read article, reaction/comment/metric, article podcast job, CDC embedding, semantic search.

Kết thúc Phase 3 – Thiết kế chi tiết cá nhân.

CHƯƠNG 4

KẾT QUẢ, THỬ NGHIỆM VÀ ĐÁNH GIÁ

Chương này tách rõ kết quả chung của nhóm và kết quả riêng của phần cá nhân: Vũ Đức Thành – Báo và podcast báo. Phần triển khai local được giữ ở mức tổng quát, còn bảng kết quả, kiểm thử và số liệu minh chứng tập trung vào cụm Article + Embedding.

4.1 Kết quả ứng dụng

4.1.1 Kết quả tổng quát cấp nhóm

Hệ thống đã được tổ chức theo kiến trúc microservices với Flutter App, API Gateway và các backend service độc lập. Các service chính có health check, tài liệu OpenAPI/Swagger, database riêng và có thể chạy local thông qua Docker Compose.

Bảng 4.1: Kết quả tổng quát cấp nhóm

Hạng mục	Kết quả đạt được	Trạng thái
Mobile app	Flutter App có các màn hình auth, home/show/episode, news/article, create AI, profile và notifications.	Đạt
API Gateway	Điều phối request, tách route public/protected, kiểm tra JWT và proxy đến service đích.	Đạt
Backend services	Identity, Content, AI, Article, Embedding và Notification được tách theo nghiệp vụ.	Đạt
Database-per-service	Mỗi service có schema/database riêng, tham chiếu chéo qua external ID thay vì foreign key vật lý.	Đạt
Event-driven	Kafka/Debezium hỗ trợ outbox email, CDC bài báo, category sync và xử lý bất đồng bộ.	Đạt
Local deployment	Docker Compose khởi động infrastructure, database init, services, gateway, Kafka UI, Redis và MinIO.	Đạt

4.1.2 Kết quả theo phạm vi cá nhân

Bảng 4.2: Kết quả cá nhân – Article + Embedding

Nội dung	Kết quả
Cụm phụ trách	Article + Embedding
Chức năng đã hoàn thành	Crawl/seed article, list/detail article, category, reaction/comment/metric, article podcast job, CDC embedding, category matching và semantic search.
Minh chứng chính	API Article/Embedding, Debezium/Kafka flow, Article DB, Embedding DB, màn hình news list/article detail.

4.2 Cài đặt và triển khai ứng dụng

4.2.1 Yêu cầu môi trường

- Docker và Docker Compose.
- Flutter SDK để chạy mobile app.
- File cấu hình môi trường cho các API key nếu chạy đầy đủ luồng AI/TTS/embedding.
- Port local chưa bị chiếm: 8080, 8081, 8082, 8083, 8084, 8085, 8087, 8088, 5433, 5434, 6379, 8090, 9000 và 9001.

4.2.2 Các bước triển khai local

1. Kiểm tra Docker Desktop đã chạy.
2. Cấu hình biến môi trường cần thiết cho Gemini/GenAI, MinIO/GCS hoặc SMTP nếu muốn chạy đầy đủ tính năng bên thứ ba.
3. Khởi động toàn bộ hệ thống backend bằng Docker Compose.
4. Chờ các database init/migration hoàn tất và các service chuyển sang trạng thái healthy.
5. Đăng ký Debezium connector nếu connector chưa được đăng ký tự động.

6. Chạy Flutter App và trở base URL về API Gateway `http://localhost:8080`
7. Kiểm tra health check, docs và các luồng demo chính.

Listing 4.1: Lệnh chạy hệ thống local

```
docker compose up --build
```

4.2.3 URL kiểm tra nhanh

Bảng 4.3: URL kiểm tra hệ thống sau khi khởi động

Mục đích	URL
Health check Gateway	<code>http://localhost:8080/healthz</code>
Tổng hợp docs	<code>http://localhost:8080/docs</code>
Route metadata	<code>http://localhost:8080/api/v1/_meta/routes</code>
Kafka UI	<code>http://localhost:8090</code>
MinIO Console	<code>http://localhost:9001</code>

4.3 Kết quả thử nghiệm cá nhân

Bảng 4.4: Bảng test case cá nhân – Article + Embedding

Nhóm chức năng	Test case	Kết quả mong đợi	Kết quả thực tế	TT
Article	Xem danh sách bài báo	Trả list có category/stats.	API trả article seed/crawl.	Pass
Article	Xem chi tiết bài báo	Tăng view count và trả detail.	Article detail hiển thị nội dung và metadata.	Pass
Article	Reaction/comment bài báo	Ghi interaction/comment theo user.	Dữ liệu được lưu qua Article Service.	Pass

Nhóm chức năng	Test case	Kết quả mong đợi	Kết quả thực tế	TT
Article Podcast	Tạo podcast job	Job queued/processing/completed và có asset khi xử lý xong.	Job được completed audio phụ thuộc provider/key.	Pass có điều kiện
Embedding	Search semantic	Trả article liên quan theo vector similarity.	Hoạt động khi embedding DB đã có vector.	Pass có điều kiện

4.4 Số liệu minh chứng trong CSDL

Bảng 4.5: Số liệu demo cá nhân – Article + Embedding

Hạng mục	Số lượng demo	Nguồn/ghi chú
News sources	3	Seed: VnExpress, Thanh Nien, Tech Crunch.
Articles	1+	Seed một bài smoke test; crawler có thể bổ sung thêm khi chạy thật.
Article stats	1+	Seed view count cho article đầu tiên.
Article categories	Theo taxonomy	Seed trong schema Article Service.
Article podcast jobs	Chốt khi demo	Phát sinh khi user chọn bài và tạo job podcast.
Embedding documents/chunks	Chốt khi demo	Phát sinh sau CDC/embedding job cho article.

4.5 Đánh giá hạn chế

4.5.1 Hạn chế chung

- Hệ thống hiện tối ưu cho demo/local, chưa hoàn thiện toàn bộ tiêu chuẩn production như autoscaling, backup, monitoring và alerting.
- Database-per-service giúp tách service rõ ràng nhưng đòi hỏi quản lý consistency bằng event, idempotency và retry/DLQ cẩn thận.

4.5.2 Hạn chế riêng của phần cá nhân

- Chất lượng search phụ thuộc chất lượng nội dung crawl, chunking và embedding model.
- Article podcast cần xử lý nền vì research, script, TTS và upload audio có độ trễ cao.
- CDC/category sync cần theo dõi idempotency và retry khi Kafka hoặc provider lỗi.

4.6 Định hướng phát triển

1. Hoàn thiện recommendation cá nhân hóa dựa trên lịch sử nghe/đọc, favorite categories và embedding similarity.
2. Bổ sung dashboard quản trị nội dung, nguồn tin, moderation comment và theo dõi chất lượng podcast tạo bằng AI.
3. Cải thiện AI pipeline với prompt versioning, evaluation, cache kết quả, multi-voice dialogue và kiểm duyệt nội dung tự động.
4. Triển khai production trên cloud với CI/CD, observability, log aggregation, backup database và secrets management.
5. Mở rộng notification sang push notification thực tế trên iOS/Android và template management cho nhiều loại email/in-app message.

4.7 Tài liệu tham khảo

1. Flutter documentation: <https://docs.flutter.dev>
2. Go documentation: <https://go.dev/doc>
3. FastAPI documentation: <https://fastapi.tiangolo.com>
4. PostgreSQL documentation: <https://www.postgresql.org/docs>

5. pgvector documentation: <https://github.com/pgvector/pgvector>
6. Apache Kafka documentation: <https://kafka.apache.org/document>
7. Debezium documentation: <https://debezium.io/documentation>
8. Docker Compose documentation: <https://docs.docker.com/compose>
9. Google GenAI/Gemini documentation.
10. Tài liệu nội bộ repository: README của từng service trong hệ thống Pody.

Kết thúc bản báo cáo cá nhân – Vũ Đức Thành – Báo và podcast báo.